Assignment 4: Self-Supervised Representation Learning for Time Series Classification

Frederik Appel Vardinghus-Nielsen

October 12, 2026

Contents

1 Part A: Understand the methods and connect paper to code **3**
1.1 The three learning ideas . . . 3
1.1.1 T-Loss (Franceschi et al., NeurIPS 2019) . . . 3
1.1.2 TS2Vec (Yue et al., AAAI 2022) . . . 3
1.1.3 TF-C (Zhang et al., NeurIPS 2022) . . . 4
1.2 Paper-to-code map . . . 4
1.2.1 T-Loss . . . 4
1.2.2 TS2Vec . . . 6
1.2.3 TF-C . . . 9
1.2.4 Following one TF-C batch through the pipeline . . . 11

2 Part B: Raw-signal baselines **14**
2.1 Protocol . . . 14
2.2 Results . . . 14
2.3 Comments . . . 14

3 Part C: Pretrain and evaluate T-Loss and TS2Vec **14**
3.1 Protocol . . . 14
3.2 Results . . . 15
3.3 Comparison with the raw-signal baselines . . . 15
3.4 Training time and representation dimensionality . . . 17

4 Part D: Robustness to contiguous sensor dropout **17**
4.1 Corruption design . . . 17
4.2 Results (100%-label models) . . . 17
4.3 Discussion . . . 17

5 Part E: Focused TS2Vec continuous masking ablation **19**
5.1 Setup . . . 19
5.2 Results . . . 19
5.3 Interpretation . . . 19

6 Part F: TF-C for industrial bearing fault classification (FD-A → FD-B) **20**
6.1 Execution record . . . 20
6.2 Classification result . . . 20
6.3 What was transferred, where labels entered, and encoder status . . . 21
6.4 Material differences between the paper's protocol and the supplied execution path 21

7 Part G: Synthesis **22**
7.1 When SSL provides a useful representation and when raw nonlinear classification remains competitive . . . 22
7.2 Whether the effect of SSL becomes clearer with only 10% labels . . . 22
7.3 Stability of conclusions across the paired seeds . . . 22
7.4 Role of dataset size, dimensionality and number of classes . . . 22
7.5 Whether continuous masking improves accuracy, robustness, both or neither . . . 22
7.6 What the TF-C bearing experiment shows about transfer between operating conditions . . . 23
7.7 Implications for future multichannel industrial sensor data . . . 23

8 Experimental rules: compliance notes **23**

9 Limitations and conclusions **24**
9.1 Limitations . . . 24
9.2 Conclusions . . . 24

10 Per-seed results **24**

1 Part A: Understand the methods and connect paper to code

1.1 The three learning ideas

1.1.1 T-Loss (Franceschi et al., NeurIPS 2019)

The general idea is that sub-series of a reference series (which may be a sub-series of a larger time series) is a positive sample, while sub-series of another series are negative samples. They use a CNN with dilation for increased receptive field. The series are left padded with zero for every convolution to maintain the length of the series. If the series are long, more time stamps have representations in the end which is fixed with a max pool.

- Positive/negative samples: a positive example x^{pos} is a sub-series of the reference series x^{ref}, which itself is randomly sampled in length and location. A sub-series of another time series is negative.
- Objective: triplet loss with one anchor, one positive, and $K \geq 1$ negative samples x_k^{neg}. The objective pushes the dot product between the representations of x^{ref} and x^{pos} up and the dot products between x^{ref} and each x_k^{neg} down, through a logistic loss. Encourages the encoder to preserve information
- Fixed length problem: the convolutional network ends with a representation $Y \in \mathbb{R}^{160 \times L}$ which is max pooled into $z \in \mathbb{R}^{160}$ and thereafter transformed through $f(x) = Wz + b$ where $W \in \mathbb{R}^{320 \times 160}$. The final dot product is therefore always between vectors of length 320.
- Classification: an SVM with a radial basis function is trained on the features with their labels now available. It is evaluated in a train/test split.

1.1.2 TS2Vec (Yue et al., AAAI 2022)

The general idea is to generate related views with masking and random overlapping segments of the representations. The hierarchical loss furthermore lets the encoder learn representations which take into account a spectrum of resolutions (granularity of the series).

- Positive/negative samples: two different views of the same time series instance (segment) are achieved through random cropping and timestamp masking. The original time series is cropped to produce two distinct, overlapping time series. After an input projection layer, these are both randomly masked at specific time stamps (single timestamp, all dimensions for multivariate TS) by setting values to zero.
- Objective: Dual loss, the sum of two distinct expressions:
  - Temporal contrastive loss which compares views of the same overlapping series at the same and different timestamps. Similarity at same timestamp between two different views is positive, representations at different timestamps, from either view, are negatives.
  - Instance-wise contrastive loss compares to other time series. Positives are two views of the same time series at the same timestamp. Negatives are both views from another time series at the same time stamp.

  The loss is calculated at coarser and coarser granularity: after each level the representations are max pooled over time with kernel size 2, until a single timestamp is left. The first level works on individual timestamps.

- Fixed length problem: only the overlapping timestamps are used after passing the views through the encoder. The loss loops over time stamps and gives a single loss for them all.
- Classification: they use instance-level representations (the whole segment of time series, not just the overlap). Instances are passed through the encoder and a max pooling is performed over all timestamps, giving a feature vector of dimension K, which is the number of representation dimensions in the last layer. SVM is trained on this K-dimensional vector for classification.

1.1.3 TF-C (Zhang et al., NeurIPS 2022)

They seek to enforce discrimination in different domains. This encourages that the time domain is robust to augmentation, the frequency domain is robust to augmentation, and the time-frequency domain enforces temporal and spectral consistency.

- Positive/negative samples:
  - Time: (1) Encodings of a time series and its augmented (jittering, scaling, time-shifts, neighborhood segments) view are similar and (2) encodings of two different time series (no augmentations) are dissimilar.
  - Frequency: (1) Encodings of the Fourier transform of a time series and its augmented view are similar and (2) encodings of two different series (no augmentations) are dissimilar.
  - Time-frequency: (1) Time series and their Fourier transform must be similar after first encoding into respective domains and thereafter encoding into common time-frequency domain, (2), augmented views are dissimilar.
- Objective: Combined loss in time-, frequency-, and time-frequency domain.
- Fixed length problem: The training datasets all contain series of same length. In the released code, every dataset is cut to its first channel and truncated to `TSlength_aligned`; nothing is padded.
- Classification: the full architecture of $z_i^T = R_T(G_T(x_i^T))$ and $z_i^F = R_F(G_F(x_i^F))$ is used and concatenated into $[z_i^T, z_i^F]$ and this is passed through a 2-layer fully connected network which projects into the number of classes needed. This classifier is jointly trained with the full pipeline on the target dataset as fine tuning.

1.2 Paper-to-code map

Verified checkout commits: T-Loss `4aff592`, TS2Vec `b0088e1`, TF-C `9667582`.

1.2.1 T-Loss

`losses/triplet_loss.py: TripletLoss.forward`

(i) Eq. (1)

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: Loss as a scalar.

(iii) `TimeSeriesEncoderClassifier.fit_encoder`

(iv) It is the loss which dictates which time series are supposed to have similar and dissimilar representations.

`networks/causal_cnn.py: CausalConvolutionBlock.forward`

(i) Figure 2(b) in the paper.

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case). This is the results after passing through the convolutional block.

(iii) `CausalCNN` where it is called for each layer in the full network.

(iv) It's the basic block which dictates the neural network architecture and therefore the inductive bias of the representation learning.

`networks/causal_cnn.py: CausalCNNEncoder.forward`

(i) Section 4 describes the full architecture. Figure 2 illustrates the convolutional blocks and dilation which are repeated in the first part of the encoder.

(ii) **Input:** tensor `(B, C, L)`, where `B` is the batch size, `C` is the number of channels, and `L` is the length of the time series (equal length case).
Output: tensor `(B, 320)`, where `B` is the batch size and 320 is the output dimension of the encoder. The time dimension L is removed with max pooling squeezing the time dimension.

(iii) `TimeSeriesEncoderClassifier.encode`

(iv) It is the full encoder: the causal convolutions, the max pooling over time and the final linear layer. No classification happens here.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.fit_encoder`

(i) Section 4 on architecture and Section S2 on hyperparameters.

(ii) **Input:** Training set X.
Output: The fitted encoder.

(iii) `TimeSeriesEncoderClassifier.fit`

(iv) It performs unsupervised/self-supervised fitting of the encoder.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.encode`

(i) Paragraph 4 and 5 in Section 4 on encoder architecture.

(ii) **Input:** The test set X.
Output: Features of X.

(iii) Depends on the context. In general `fit()` calls it during training to use the final encoder to produce features before training SVM, while `score()` calls it during testing to produce features for SVM classification.

- `fit_encoder()` calls it after each training epoch if certain conditions for early stopping are present. This doesn't make sense for fully self-supervised learning, so labels must be provided.

- `fit()` calls it to encode the training set once training is done such that SVM can be trained on the training features.
- `encode_window()` calls it to encode a test set on a sliding window. Useful if temporal features are needed for regression/forecasting.
- `predict()` and `score()` call it to encode a set and then perform prediction or calculate score of classification. Mostly used for testing.

(iv) It uses the final encoder (unless used during early stopping) to encode features from either train or test sets before classification or prediction.

Required specifics

- How `fit_encoder` selects the fixed-length or varying-length loss: It checks whether a `numpy` sum over the training set yields `nan`s in some entries, which then indicates unequal lengths.
- How `TripletLoss.forward` samples reference, positive and negative intervals:
  - Reference/anchor: each time series is used as reference once per epoch. The length is chosen at random between the length of the positive example and the length of the full series. The position is chosen to overlap the positive example.
  - Negative: a specified number of random time series are chosen as negatives for each anchor. Negatives are drawn at random from the whole training set, so a negative can come from the anchor's own series.
  - Positive: chosen as a random sub-series of each anchor. In practice the length and position of the positive is chosen first.
- How `CausalCNNEncoder` produces one fixed-length vector: it uses AdaptiveMaxPool1d to max pool over the time dimension. This produces a feature vector which only has length one in the remaining "time" dimension.
- How to call `fit_encoder` without labels: call it without passing training labels `y`. It defaults to none and performs unsupervised learning without early stopping.

1.2.2 TS2Vec

`ts2vec.py: TS2Vec.fit`

(i) Section 2.3 on Contextual Consistency which describes the timestamp masking and random cropping which serve as the augmentations for robustness, and Section 2.4 on Hierarchical Contrasting which is described in Algorithm 1. The whole loss is shown in Equations (1), (2), and (3).

(ii) **Input:** Train data, number of epochs to train, maximum number of iterations (batches) to train, and whether to print training loss after each epoch.
Output: Log of losses during training.

(iii) It's called in `train.py`.

(iv) It contains the central training loop which includes cropping, augmentation, and loss.

`ts2vec.py: TS2Vec._eval_with_pooling`

(i) Sections 3.1, 3.2, and 3.3 on applications. The evaluation/representation of the time series varies depending on the downstream task.

(ii) **Input:** Test data `x` on which to evaluate the averaged model, whether to use a mask, whether to slice.
Output: Representations from running the encoder and the chosen pooling strategy.

(iii) `TS2Vec.encode()`

(iv) It applies the trained model to a test set and the pooling strategy which is chosen based upon the downstream task.

`ts2vec.py: TS2Vec.encode`

(i) Section 2.2 on Model Architecture and Sections 3.1, 3.2, and 3.3 on downstream tasks.

(ii) **Input:** Test data to calculate representations from. Masking, sliding length and padding for representation altering, encoding window for restriction before encoding, causal, and batch sizes are additional inputs.
Output: The representations from encoding the test data.

(iii) `anomaly_detection.py`, `classification.py`, and `forecasting.py` for the three tasks, respectively.

(iv) Uses the fitted encoder to encode test data for use in downstream task.

`models/losses.py: hierarchical_contrastive_loss`

(i) Equations (1)–(3); its `while` loop is the loop in Algorithm 1.

(ii) **Input:** Two different representation of the same batch to calculate loss between.
Output: Loss between the representations as defined by Equation (3).

(iii) `TS2Vec.fit()`

(iv) Calculates loss which dictates training.

`models/losses.py: instance_contrastive_loss`

(i) Equation (2).

(ii) **Input:** Two representations of the same batch after encoding during training.
Output: The instance contrastive loss between the two representations. Two representations at the same timestamp are positive, while same representation of two different time series are negative.

(iii) `hierarchical_contrastive_loss`.

(iv) Forces the model to have representations of the same series in different contexts and with different masks to be similar, while keeping representations of other series dissimilar.

`models/losses.py: temporal_contrastive_loss`

(i) Equation (1).

(ii) **Input:** Two representations of the same batch after encoding during training.
Output: The temporal contrastive loss between the two representations. The same timestamp for the two representations is a positive pair, while different timestamps in both the same and the other representation are negative pairs.

(iii) `hierarchical_contrastive_loss`.

(iv) Encourages the model to discriminate representations over time. Representations at the same timestamps should have similar representations while temporal differences should be reflected in different representations.

`models/encoder.py: TSEncoder.forward`

(i) Section 2.2, paragraphs 2 and 3.

(ii) **Input:** Data to be encoded. It has been cropped and augmented before.
Output: Encoded batch.

(iii) `TS2Vec.fit()` and `_eval_with_pooling`.

(iv) Contains the convolutional part of the model architecture.

`models/encoder.py: generate_binomial_mask`

(i) The paragraph **Timestamp Masking** in Section 2.3.

(ii) **Input:** `B`, `T`, and `p`; batch size, time length, and masking probability, respectively.
Output: A boolean tensor with shape `(B, T)` with values `False` and `True` indicating masking or not, respectively.

(iii) `TSEncoder.forward()`

(iv) Performs the timestamp masking which forces robustness and context understanding of the model.

`models/encoder.py: generate_continuous_mask`

(i) Not mentioned in article.

(ii) **Input:** `B`, `T`, `n`, and `l`; batch size, time length, number of masked sequences, and length of masked sequences as a fraction of full length, respectively.
Output: A boolean tensor with shape `(B, T)` with values `False` and `True` indicating masking or not, respectively.

(iii) `TSEncoder.forward()`

(iv) Creates continuous runs of masking. They may overlap.

Required specifics

- Overlapping crops in `TS2Vec.fit`: They are created by in sequence determining the length of the overlap of the crops, the start and end of the overlap, the extension of crops 1 to the left, and finally the extension of crop 2 to the right.
- Where timestamp masking is applied: In TSEncoder.forward(). This is only applied during training.
- Mapping of the two elementary losses and hierarchical pooling to Equations 1–3 and Algorithm 1: `temporal_contrastive_loss`, `instance_contrastive_loss`, and `hierarchical_contrastive_l` match Equations (1)–(3), respectively.
- The `TS2Vec.encode` option returning one vector per complete series: Set the argument `encoding_window="full_series"`. This causes a max pool of the features over all time stamps after encoding.

1.2.3 TF-C

`code/TFC/dataloader.py: Load_Dataset.__init__`

(i) Sections 4.1 and 4.2 on data augmentation in time and frequency domains.

(ii) **Input:** A dataset with time series and labels, a config, training mode, max size of a debugging dataset, and whether to debug (use a smaller dataset).
Output: No output. Saves two versions of the time series: the augmented version and the Fourier transformed and then augmented.

(iii) Three times in `data_generator`; once for each of training, fine-tuning, and testing.

(iv) Performs the steps of loading and augmenting data before it is passed through the first set of encoders.

`code/TFC/dataloader.py: Load_Dataset.__getitem__`

(i) Has no direct counterpart in the article. It just serves to pop out a single time series, its augmentations, and transforms.

(ii) **Input:** Index of the requested time series.
Output: The requested time series, its augmentation in time, the Fourier transform, its augmentation in frequency, and the labels. If run at fine tuning or test time, the augmentations are the same as the original representations.

(iii) `model_pretrain`, `model_finetune`, and `model_test`.

(iv) Is a wrapper for providing the augmented and transformed views and labels during training, fine tuning, and test.

`code/TFC/dataloader.py: data_generator`

(i) No direct counterpart. Paragraph 4 of Section 1 explains the concept of pre-training, fine-tuning, and then testing. This split is done by this generator.

(ii) **Input:** Path to the source data, path to the target (fine tuning and test dataset), configs, training mode (pre-train, fine tuning, or test), and whether to run in debugging mode.
Output: Loaders for the three different cases and datasets: training, fine tuning, and testing.

(iii) `main.py`

(iv) Helps loading the correct datasets with the corresponding augmentations for pre-training, fine tuning, and testing.

`code/TFC/dataloader.py: the fft.fft(...).abs() operation`

(i) Section 4.2, paragraph 1, where the frequency representation is described as the Fourier transformation of the time series.

(ii) **Input:** The whole dataset of time series.
Output: The absolute values of the Fourier transform of the time series.

(iii) It is saved in the class instance as the spectral representation of the time series. During training, it is computed once in `Load_Dataset.__init__`, and `TFC.forward()` only sees the result.

(iv) Is responsible for the spectral representation which the authors deem valuable for time series representation.

`code/TFC/augmentations.py: DataTransform_TD, DataTransform_FD, remove_frequency, add_frequency`

(i) Section 4.2, paragraph 2 on augmentations of the Fourier transform of the time series.

(ii) **Input:** Fourier transform of a single time series.
Output: An augmented Fourier transform where frequencies are added and removed at random.

(iii) `Load_Dataset.__init__()` when augmenting the Fourier transformed time series.

(iv) Performs augmentations of the absolute Fourier transform of the raw time series in order to provide positive pairs in the spectral representations. Because the two augmented copies are summed, no frequency is actually removed (see the frequency-domain answer below).

`code/TFC/model.py: TFC.__init__, TFC.forward, target_classifier.forward`

(i) No particular part of the article which corresponds to this.

(ii) **Input:** Training data and its Fourier transform.
Output: The four representations encoded by the various combinations of the encoders.

(iii) `TFC.__init__` in `main.py`; `TFC.forward` in `model_pretrain`, `model_finetune` and `model_test`; `target_classifier.forward` in `model_finetune` and `model_test`.

(iv) Calls and structures the overall architecture and data flow in the model. `init` defines the two encoders and the two projectors, `forward` defines the four representations when the time series, the Fourier transforms, and their augmentations are passed through the encoders, and `target_classifier.forward` is simply a small, fully connected neural network which is trained during fine tuning and used for classification during testing.

`code/TFC/loss.py: NTXentLoss_poly.forward`

(i) Equations (1), (2), and (3).

(ii) **Input:** Representations Z and $\tilde{Z}$.
Output: Total loss from positive and negative samples in the two representations.

(iii) It is called in `trainer.py: model_pretrain` and `model_finetune` for use during training and fine tuning.

(iv) Implements the well known NT-Xent loss between the various positive and negative samples in the model.

`code/TFC/trainer.py: Trainer, model_pretrain, model_finetune, model_test`

(i) Sections 5.1 and 5.2 on results from pre-training and fine-tuning.

(ii) **Input:** Model, optimizer, classifier, classifier optimizer, dataloaders, and configuration files.
Output: No returns. Prints logging during training or fine tuning and saves models and results.

(iii) `main.py`

(iv) `Trainer` splits the model in pre-training and fine-tuning and runs the overall logic for pre-training and fine-tuning and saves the models, `model_pretrain` runs the pre training pipeline and returns the loss, `model_finetune` trains the encoder and classifier on the FD-B training set, and `model_test` evaluates them on the FD-B test set.

`code/TFC/main.py: the subset setting, call to data_generator, pretrained-checkpoint loading`

(i) No directly corresponding parts of the article.

(ii) **Input:** Not a function, so no inputs. However, needs configuration files and datasets.
Output: No outputs but logs progress during pre-training or fine-tuning. The models which are trained are saved to the local machine during this function.

(iii) No immediate caller.

(iv) Top level file for entry for both pre-training and fine-tuning.

1.2.4 Following one TF-C batch through the pipeline

1. `Load_Dataset.__init__` holds the time series `x_data` of shape `(N, 1, 5120)` and computes, once for the whole dataset, the spectrum `x_data_f = fft(x_data).abs()` and the augmented views `aug1` (jitter) and `aug1_f` (frequencies removed and added), all of the same shape.

2. The loader in `model_pretrain` yields a batch `data`, `aug1`, `data_f`, `aug1_f`, each `(64, 1, 5120)`, and the labels, which are not used in pretraining.

3. `model(data, data_f)` calls `TFC.forward`: the time encoder gives h^T of shape `(64, 5120)` and the time projector z^T of shape `(64, 128)`; the frequency encoder and projector give h^F and z^F of the same shapes. A second call `model(aug1, aug1_f)` gives the same four tensors for the augmented views.

4. `NTXentLoss_poly` gives `loss_t` from $(h^T, \tilde{h}^T)$, `loss_f` from $(h^F, \tilde{h}^F)$ and `l_TF` from (z^T, z^F). The optimised loss is `0.2*(loss_t + loss_f) + l_TF`, followed by `loss.backward()` and an optimiser step.

5. In fine-tuning and testing, z^T and z^F are concatenated to `(B, 256)` and passed to `target_classifier`: a linear layer to 64, a sigmoid, and a linear layer to the 3 classes.

FFT dimension, spectrum conversion, `fft` vs. `rfft`

- **FFT dimension:** over the time dimension, two sided to keep matching length with the time domain part.
- **Spectrum conversion:** absolute value/magnitude is used.
- `fft` or `rfft`: `fft` is used. This keeps the same length as the original time series.

Active time-domain augmentation; the two frequency operations and how they are combined

- **Time domain:** the paper mentions a bank of multiple different augmentations of which a random one should be chosen. In practice only jitter is used.
- **Frequency domain:** Frequencies (bins) are added and removed at random. The two augmented copies are summed, so kept bins are doubled while the 'removed' bins stay at their original value: removal only halves a bin relative to the rest.

Attributes defining the time encoder, frequency encoder and the two projectors; the four outputs of `TFC.forward` in terms of h and z

- **Time and frequency encoders (same parameters):** `transformer_encoder_t` and `transformer_encoder_f`, defined by the length of the time series, dimension of the intermediate layer in the transformer, and number of heads of the attention mechanism.
- **Projectors (same parameters):** `projector_t` and `projector_f`, defined by the dimension of the first linear layer, dimension of batch normalisation, and dimensions of last linear layer.
- **Relation of outputs to paper notation:**
  - `h_time` corresponds to $\mathbf{h}^T$ which is the encoding of the time series by the time encoder, utilizing transformer architecture.
  - `z_time` corresponds to $\mathbf{z}^T$ which is the projection of $\mathbf{h}^T$ by the time projector, utilizing linear layers and ReLU.
  - `h_freq` corresponds to $\mathbf{h}^F$ which is the encoding of the Fourier transform of the time series by the frequency encoder, utilizing transformer architecture.
  - `z_freq` corresponds to $\mathbf{z}^F$ which is the projection of $\mathbf{h}^F$ by the frequency projector, utilizing linear layers and ReLU.

Loss terms in `model_pretrain` vs. Equations 1–4

- `loss_t = nt_xent_criterion(h_t, h_t_aug)` is the within-time loss from Equation (1).
- `loss_f = nt_xent_criterion(h_f, h_f_aug)` is the within-frequency loss from Equation (2).

- `l_TF = nt_xent_criterion(z_t, z_f)` is the loss between the projection from the frequency- and time-encoders, noted as S_i^{TF} in Equation (3)
- `l_1, l_2`, and `l_3` are the negative time-frequency consistency losses, noted as S_i^{pair} in Equation (3).
- `loss = lam*(loss_t + loss_f) + l_TF` is the total loss, which should correspond to Equation (4), but `l_1, l_2`, and `l_3` are missing.

Tensors concatenated for the downstream classifier; is the encoder frozen or fine-tuned?

- During fine tuning the representations z_T and z_F are concatenated and used as features for the classifier. The encoder is fine-tuned, and the classifier is trained.
- During testing the representations z_T and z_F are concatenated and used as features for the classifier. Both the encoder and classifier are frozen.

`main.py` → `data_generator` → `Trainer` **for FD-A → FD-B**

- Because `subset = True`, pretraining uses 640 of the 8,184 FD-A series (10 batches of 64). Fine-tuning uses all 60 FD-B training series and testing all 13,559 FD-B test series.
- `FD_B/val.pt` is never loaded, only `train.pt` and `test.pt` are loaded.
- The test loader is evaluated once per fine tuning epoch.
- The test influences the best results reported, since the results reported are those which resulted in the best test.

Discrepancies Discrepancies from the paper's stated protocol are recorded, not repaired.

- Consistency loss is excluded from total loss. This is the paper's central claim missing.
- The weight λ of losses varies from training to tuning.
- They use poly loss instead of pure NT-Xent.
- Frequency augmentations are summed instead of randomly selected.
- The paper says that only one frequency bin is augmented, the code augments 10%.
- Only jittering augmentation is used in the time domain. There is no augmentation bank.
- Perturbations of the Fourier transform are not mirrored and the spectrum is therefore not obtainable from a real signal.
- The first set of encoders are specified as 3-layer 1-D ResNets in the paper, while they are 2-layer transformers in the code.
- All datasets are reduced to one channel and truncated to `TSlength_aligned`, which is 5120 for FD-A and FD-B (178 is the SleepEEG value).

Device-specific tensor construction in `remove_frequency` and `add_frequency` `torch.cuda.FloatTenso` generates a tensor on the GPU. If no GPU is on the machine on which the code is run, the script will crash.

2 Part B: Raw-signal baselines

2.1 Protocol

- Scaling: applied per channel before flattening, across time series and fitted on the full training set.
- Tuning of classifiers/probes: stratified cross-validation on training labels to ensure close to even splits across the train/val barrier. Tuning of C for logistic regression, C and γ for RBF-SVM.
- AWR class problem: AWR has too few instances in each class to ensure at least one of each on either side of the train/val barrier under 10% label usage. The probes use `sklearn` defaults in that case.

2.2 Results

The results are shown in Table 1.

Dataset	Model	Labels	Accuracy	Macro-F1
FordA	Logistic regression	100%	0.4922 ± 0.0034	0.4908 ± 0.0046
FordA	Logistic regression	10%	0.5058 ± 0.0083	0.5057 ± 0.0084
FordA	RBF-SVM	100%	0.8280 ± 0.0000	0.8278 ± 0.0000
FordA	RBF-SVM	10%	0.6689 ± 0.0131	0.6674 ± 0.0137
AWR	Logistic regression	100%	0.9733 ± 0.0000	0.9732 ± 0.0000
AWR	Logistic regression	10%	0.8278 ± 0.0222	0.8200 ± 0.0266
AWR	RBF-SVM	100%	0.9800 ± 0.0000	0.9800 ± 0.0000
AWR	RBF-SVM	10%	0.2300 ± 0.1305	0.2181 ± 0.1576
Epilepsy	Logistic regression	100%	0.6329 ± 0.0167	0.5996 ± 0.0183
Epilepsy	Logistic regression	10%	0.4396 ± 0.0715	0.3744 ± 0.0881
Epilepsy	RBF-SVM	100%	0.8478 ± 0.0000	0.8369 ± 0.0000
Epilepsy	RBF-SVM	10%	0.5145 ± 0.1007	0.4799 ± 0.1223

Table 1: Raw-signal baselines. Test accuracy and macro-F1 as mean ± standard deviation over three paired seeds.

2.3 Comments

3 Part C: Pretrain and evaluate T-Loss and TS2Vec

3.1 Protocol

- Pretraining: on the full unlabelled training split with the published default hyperparameters; 1500 steps for T-Loss and 600 iterations for TS2Vec (200 on Epilepsy, by its own rule). One vector per series, using `encoding_window="full_series"` for TS2Vec. Training is seeded but not bitwise reproducible on GPU.
- Seeds: 42, 123 and 456.
- Hyperparameter grids: $C \in \{0.01, 0.1, 1, 10, 100\}$ for logistic regression; $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\text{scale}, 0.001, 0.01, 0.1\}$ for RBF-SVM.
- Folds: 5-fold stratified cross-validation, reduced to the size of the smallest class when that is lower.

Dataset	Logistic regression	RBF-SVM
FordA	Bad, since it assumes that the same timestamps are comparable across series, but they are not phase-synced. Equivalent to random guessing.	Performs well because the RBF-SVM merely uses neighborhoods to classify. If it has seen a set of phase-shifted time series in the same class it will be able to classify a new one well.
AWR	Performs well because intra-class time series are similar and synchronous.	Same as logistic regression with 100% labels. With 10% labels it drops to 0.23: no tuning is possible with about one label per class, so it runs on default hyperparameters.
Epilepsy	Performs poorly because, as on FordA, the waveform has no consistent position in time. It likely beats chance mainly because the mean level of each channel differs between classes.	Likely performs well mainly because the classes differ in amplitude, which distance captures and a linear model on the raw signal cannot. With only 13 labelled series at 10% it loses most of that.

Table 2: Comments on the raw-signal baselines, per dataset and probe.

- Fixed defaults: when a class has a single labelled example, no tuning is possible and the `sklearn` defaults are used ($C = 1$, $\gamma = \text{scale}$).
- Representation scaling: the representations are standardised per dimension, fitted on the training representations, before the probe. Part B has no equivalent step, and TS2Vec's own protocol scales only before its logistic regression.

Principal representation test The probe treated as the principal representation test is logistic regression, since it is the linear probe.

3.2 Results

The results are shown in Table 3.

3.3 Comparison with the raw-signal baselines

- Both methods improve clearly on FordA (from chance to 0.92) and Epilepsy (from 0.63 to 0.97), but not on AWR, where T-Loss is slightly below and TS2Vec slightly above the raw baseline.
- The representations are close to linearly separable: a linear classifier that fails on the raw signal reaches 0.92–0.97 on them. The performance is better than RBF-SVM on FordA and Epilepsy.
- T-Loss and TS2Vec are on par. The differences on FordA and Epilepsy are within the spread across seeds; only on AWR with 100% labels is TS2Vec ahead (0.983 against 0.966).
- AWR is already saturated on the raw signal and becomes less stable with 10% labels, where only 27 labelled series cover 25 classes. Compared to RBF-SVM, it is more stable when using fewer labels, but neither probe could be tuned there: with about one label per class both run on default hyperparameters, which the RBF-SVM tolerates far worse than logistic regression.

Dataset	Method	Probe	Labels	Accuracy	Macro-F1
FordA	T-Loss	Logistic regression	100%	0.9230 ± 0.0102	0.9229 ± 0.0102
FordA	T-Loss	Logistic regression	10%	0.9098 ± 0.0055	0.9096 ± 0.0054
FordA	T-Loss	RBF-SVM	100%	0.9293 ± 0.0031	0.9292 ± 0.0030
FordA	T-Loss	RBF-SVM	10%	0.9152 ± 0.0085	0.9150 ± 0.0086
FordA	TS2Vec	Logistic regression	100%	0.9273 ± 0.0077	0.9272 ± 0.0077
FordA	TS2Vec	Logistic regression	10%	0.9121 ± 0.0053	0.9119 ± 0.0052
FordA	TS2Vec	RBF-SVM	100%	0.9348 ± 0.0072	0.9348 ± 0.0072
FordA	TS2Vec	RBF-SVM	10%	0.9169 ± 0.0052	0.9167 ± 0.0051
AWR	T-Loss	Logistic regression	100%	0.9656 ± 0.0077	0.9651 ± 0.0081
AWR	T-Loss	Logistic regression	10%	0.8144 ± 0.0506	0.8012 ± 0.0607
AWR	T-Loss	RBF-SVM	100%	0.9700 ± 0.0058	0.9697 ± 0.0057
AWR	T-Loss	RBF-SVM	10%	0.1800 ± 0.0416	0.1761 ± 0.0366
AWR	TS2Vec	Logistic regression	100%	0.9833 ± 0.0033	0.9832 ± 0.0034
AWR	TS2Vec	Logistic regression	10%	0.8378 ± 0.0271	0.8252 ± 0.0271
AWR	TS2Vec	RBF-SVM	100%	0.9833 ± 0.0058	0.9833 ± 0.0056
AWR	TS2Vec	RBF-SVM	10%	0.1578 ± 0.0453	0.1483 ± 0.0555
Epilepsy	T-Loss	Logistic regression	100%	0.9686 ± 0.0084	0.9684 ± 0.0083
Epilepsy	T-Loss	Logistic regression	10%	0.7440 ± 0.0233	0.7406 ± 0.0252
Epilepsy	T-Loss	RBF-SVM	100%	0.9734 ± 0.0042	0.9738 ± 0.0043
Epilepsy	T-Loss	RBF-SVM	10%	0.7005 ± 0.0633	0.6940 ± 0.0636
Epilepsy	TS2Vec	Logistic regression	100%	0.9662 ± 0.0084	0.9647 ± 0.0091
Epilepsy	TS2Vec	Logistic regression	10%	0.7367 ± 0.0493	0.7336 ± 0.0474
Epilepsy	TS2Vec	RBF-SVM	100%	0.9565 ± 0.0000	0.9548 ± 0.0005
Epilepsy	TS2Vec	RBF-SVM	10%	0.6546 ± 0.0834	0.6589 ± 0.0704

Table 3: Frozen-representation classification. Mean $\pm$ standard deviation over three paired seeds.

- The drop from 100% to 10% labels follows the number of labels left: about 0.014 on FordA (360 series), 0.15 on AWR (27) and 0.23 on Epilepsy (13). On AWR and Epilepsy it is no smaller than for the raw baseline.

3.4 Training time and representation dimensionality

Training time and representation dimensionality are shown in Table 4.

Dataset	Method	Pretraining time	Representation dimension
FordA	T-Loss	638 ± 7 s	320
FordA	TS2Vec	29 ± 7 s	320
AWR	T-Loss	547 ± 38 s	320
AWR	TS2Vec	18 ± 1 s	320
Epilepsy	T-Loss	545 ± 12 s	320
Epilepsy	TS2Vec	7 ± 0 s	320

Table 4: Pretraining time per encoder and representation dimensionality. Mean ± standard deviation over the three seeds on one NVIDIA RTX 4500 Ada Generation GPU; T-Loss runs 1500 steps, TS2Vec 600 iterations (200 on Epilepsy).

The T-Loss timings for FordA seeds 42 and 123 come from a second, timing-only training run, since the first runs were not timed. The encoders used for the results are the original ones.

4 Part D: Robustness to contiguous sensor dropout

4.1 Corruption design

- Proportion: 20% of each series (100 timesteps on FordA, 41 on Epilepsy, 29 on AWR).
- Intervals: one contiguous interval per series, modelling a single sensor outage.
- Position: the start is drawn uniformly at random per series, so that no fixed region is systematically removed.
- Channels: all channels lose the same interval. Removing a single channel would cost AWR 2.2% of its data against 20% for FordA.
- Replacement: the training mean of the channel, which is exactly 0 after scaling. The removed part then contributes nothing to a linear classifier on the raw signal.
- Scope: applied to the test set only. The models never see corrupted data during training.
- Seeding: the corruption is seeded by the run's seed, so every method sees identical corrupted inputs.

4.2 Results (100%-label models)

Accuracy is shown in Table 5 and macro-F1 in Table 6.

4.3 Discussion

Only two of three dimensions requested can be evaluated here since only one preprocessing convention was used.

- All results for the corrupted cases are lower, except the results from logistic regression for FordA, which is at chance in both cases, so its +0.014 is noise.

Dataset	Method	Clean	Corrupted	Δ
FordA	Logistic regression (raw)	0.4922 ± 0.0034	0.5063 ± 0.0129	$+0.0141 \pm 0.0097$
FordA	RBF-SVM (raw)	0.8280 ± 0.0000	0.8058 ± 0.0042	-0.0222 ± 0.0042
FordA	T-Loss	0.9230 ± 0.0102	0.8937 ± 0.0081	-0.0293 ± 0.0054
FordA	TS2Vec	0.9273 ± 0.0077	0.8997 ± 0.0052	-0.0275 ± 0.0123
AWR	Logistic regression (raw)	0.9733 ± 0.0000	0.9633 ± 0.0033	-0.0100 ± 0.0033
AWR	RBF-SVM (raw)	0.9800 ± 0.0000	0.9700 ± 0.0067	-0.0100 ± 0.0067
AWR	T-Loss	0.9656 ± 0.0077	0.9511 ± 0.0158	-0.0144 ± 0.0084
AWR	TS2Vec	0.9833 ± 0.0033	0.9611 ± 0.0102	-0.0222 ± 0.0135
Epilepsy	Logistic regression (raw)	0.6329 ± 0.0167	0.5966 ± 0.0327	-0.0362 ± 0.0377
Epilepsy	RBF-SVM (raw)	0.8478 ± 0.0000	0.8116 ± 0.0126	-0.0362 ± 0.0126
Epilepsy	T-Loss	0.9686 ± 0.0084	0.9396 ± 0.0233	-0.0290 ± 0.0192
Epilepsy	TS2Vec	0.9662 ± 0.0084	0.9517 ± 0.0182	-0.0145 ± 0.0126

Table 5: Accuracy: clean versus contiguous-dropout test performance for the 100%-label models. T-Loss and TS2Vec are evaluated with logistic regression, the principal probe. Mean ± standard deviation over three paired seeds; Δ is corrupted minus clean.

Dataset	Method	Clean	Corrupted	Δ
FordA	Logistic regression (raw)	0.4908 ± 0.0046	0.5054 ± 0.0142	$+0.0145 \pm 0.0100$
FordA	RBF-SVM (raw)	0.8278 ± 0.0000	0.8044 ± 0.0040	-0.0234 ± 0.0040
FordA	T-Loss	0.9229 ± 0.0102	0.8934 ± 0.0083	-0.0295 ± 0.0055
FordA	TS2Vec	0.9272 ± 0.0077	0.8997 ± 0.0051	-0.0275 ± 0.0123
AWR	Logistic regression (raw)	0.9732 ± 0.0000	0.9632 ± 0.0033	-0.0100 ± 0.0033
AWR	RBF-SVM (raw)	0.9800 ± 0.0000	0.9701 ± 0.0065	-0.0099 ± 0.0065
AWR	T-Loss	0.9651 ± 0.0081	0.9507 ± 0.0158	-0.0144 ± 0.0080
AWR	TS2Vec	0.9832 ± 0.0034	0.9609 ± 0.0103	-0.0223 ± 0.0136
Epilepsy	Logistic regression (raw)	0.5996 ± 0.0183	0.5719 ± 0.0376	-0.0277 ± 0.0402
Epilepsy	RBF-SVM (raw)	0.8369 ± 0.0000	0.8062 ± 0.0126	-0.0307 ± 0.0126
Epilepsy	T-Loss	0.9684 ± 0.0083	0.9407 ± 0.0224	-0.0277 ± 0.0185
Epilepsy	TS2Vec	0.9647 ± 0.0091	0.9517 ± 0.0174	-0.0130 ± 0.0115

Table 6: Macro-F1: clean versus contiguous-dropout test performance for the 100%-label models. T-Loss and TS2Vec are evaluated with logistic regression, the principal probe. Mean ± standard deviation over three paired seeds; Δ is corrupted minus clean.

- Representation: using the encoded representation instead of the raw signals is not more robust – they drop at about the same rate (the differences in Δ are within the seed spread), but the representations keep a much higher absolute accuracy on FordA and Epilepsy.
- Classifier: no difference in robustness. With the RBF-SVM on the same representations the drops on FordA are -0.029 (T-Loss) and -0.025 (TS2Vec), against -0.029 and -0.028 with logistic regression. The full rows are in `results/metrics/corruption.csv`.
- Preprocessing: not tested, since only one fill was used. The fill is 0 after scaling, the mildest choice for a linear classifier on the raw signal, so the raw baselines are not disadvantaged by it.

5 Part E: Focused TS2Vec continuous masking ablation

5.1 Setup

- Masks: both at the checkout's defaults. Binomial masks each timestamp independently with $p = 0.5$; continuous masks 5 runs of 10% of the crop each, which may overlap (about 41% in expectation).
- Matched: encoder, 600 iterations, seeds (and thereby initial weights), probe, label regime and corrupted test sets. The binomial-mask encoders are the ones trained in Part C; only the continuous-mask encoders are trained here.
- Departure: `TS2Vec` does not expose `mask_mode`, so it is set on the encoder after construction. The checkout is not modified.
- Confounds: the amount masked differs (about 41% against 50%), and the training mask zeroes hidden vectors while the test corruption replaces input values.

5.2 Results

The results are shown in Table 7.

Masking	Metric	Clean	Corrupted	Δ
Binomial (default)	Accuracy	0.9273 ± 0.0077	0.8997 ± 0.0052	-0.0275 ± 0.0123
Continuous	Accuracy	0.9278 ± 0.0084	0.8972 ± 0.0032	-0.0306 ± 0.0087
Binomial (default)	Macro-F1	0.9272 ± 0.0077	0.8997 ± 0.0051	-0.0275 ± 0.0123
Continuous	Macro-F1	0.9277 ± 0.0085	0.8971 ± 0.0031	-0.0305 ± 0.0088

Table 7: FordA: standard versus continuous TS2Vec masking, clean and contiguous-dropout test sets, with logistic regression on 100% labels. Mean $\pm$ standard deviation over three paired seeds; Δ is corrupted minus clean.

5.3 Interpretation

- Clean performance: no big difference between binomial and continuous (0.9273 against 0.9278).
- Corrupted performance: no big difference (0.8997 against 0.8972).
- Sensitivity to corruption relative to clean score: no big difference (-0.028 against -0.031). All three differences are well within the seed spread of about 0.01.

6 Part F: TF-C for industrial bearing fault classification (FD-A → FD-B)

6.1 Execution record

The execution record is shown in Table 8.

Item	Value
Commands (pretraining)	`python main.py --training_mode pre_train --pretrain_dataset FD_A --target_dataset FD_B --seed 42 --logs_save_dir results/tfc`, run from `code/TFC`
Commands (fine-tuning / testing)	The same command with `--training_mode fine_tune_test`
Checkout	Commit `9667582`, no tracked file modified
Environment (Python, PyTorch, OS)	Python 3.13.11, PyTorch 2.11.0 (CUDA 12.8), NumPy 2.4.2, scikit-learn 1.8.0, Windows 11
Hardware	NVIDIA RTX 4500 Ada Generation, 25.8 GB
Seed	42
Data actually consumed	Pretraining: 640 of the 8184 FD-A training windows. Fine-tuning: all 60 FD-B training windows. Testing: all 13559 FD-B test windows, after each of the 40 epochs. The validation files are never loaded.
Training time	Pretraining 180 s, fine-tuning and testing 249 s
Checkpoint used	`ckp_last.pt`, the model after the last (40th) pretraining epoch

Table 8: TF-C FD-A → FD-B execution record.

Both stages were started with `python dev/9_tfc_fda_fdb.py`, which runs the unmodified `main.py` and stores the record in `results/tfc/execution_record.json`. Three things were added from outside the checkout:

- The alias `np.float = float` is set in memory before `main.py` starts, since `trainer.py` uses it and it was removed in NumPy 1.24.
- `datasets/FD_A` and `datasets/FD_B` are directory links to the downloaded files, since `main.py` reads from these fixed paths.
- `--logs_save_dir` sends the logs and the pretraining checkpoint to `results/tfc`.

6.2 Classification result

The classification result is shown in Table 9.

Run	Accuracy	Macro-F1
This run (FD-B test), one seed	0.4553	0.2086
TF-C supplement, Table 5	0.8934 ± 0.0379	0.9162 ± 0.0826

Table 9: FD-B classification result, compared with Table 5 of the TF-C supplement. The value for this run is the one the supplied code reports: the epoch with the highest test accuracy.

The run does not reproduce Table 5.

- The fine-tuned model predicts the same class for every test window. The FD-B test set has class shares 8.9%, 45.5% and 45.5%, so this gives accuracy 0.4553 and macro-recall 0.3333.
- Accuracy and macro-F1 are identical in all 40 epochs, so choosing the best test epoch has no effect here.

- The result is close to the non-deep-learning KNN baseline in Table 5 (accuracy 0.4473, F1 0.2284) and below the randomly initialised model (0.4736, 0.4911).

The cause is that the encoders collapse during pretraining, before any FD-B label is used.

- The pretraining loss settles at 8.18. This is the value of the loss when all embeddings are identical: $1.4\,(\ln 127 + 1) \approx 8.18$ for a batch of 64.
- With identical embeddings the classifier output is constant, which gives the single predicted class.

The conclusion is limited to the supplied execution path.

- It shows that the released code at commit `9667582` does not learn on FD-A $\rightarrow$ FD-B.
- It does not show that the result in the paper is wrong or that the time-frequency consistency idea fails, since the released code differs from the protocol in the paper (see below).
- It was not tested whether the code works with the library versions from 2022.

6.3 What was transferred, where labels entered, and encoder status

- Transferred from FD-A: all weights of the TF-C model after pretraining, i.e. the time encoder, the frequency encoder and the two projectors. The classifier is new and randomly initialised.
- FD-B labels enter in fine-tuning through the cross-entropy loss on the 60 labelled training windows. The same labels select which fine-tuned model is kept (highest F1 on the training batch) and fit the KNN classifier.
- FD-B test labels are used to pick the reported epoch.
- The encoder is updated, not frozen: fine-tuning optimises all encoder and projector weights together with the classifier, and the contrastive losses stay in the fine-tuning objective.

6.4 Material differences between the paper's protocol and the supplied execution path

Based on the Part A inspection:

- Subset use: `main.py` sets `subset = True`, so pretraining uses 640 of the 8184 FD-A windows (7.8%). This is not a command-line option.
- Loss composition: the consistency loss L_C is computed but not included in the optimised loss, which is $0.2\,(L_t + L_f) + L_{TF}$ in pretraining. An extra poly-loss term is added to every contrastive loss.
- Validation use: the validation files are never loaded. The fine-tuned model is selected by F1 on the training batch.
- Test-set access: the test set is evaluated after every fine-tuning epoch, and the reported result is the epoch with the highest test accuracy.

7 Part G: Synthesis

7.1 When SSL provides a useful representation and when raw nonlinear classification remains competitive

The tested SSL methods improve logistic regression on the FordA and Epilepsy datasets beyond the performance of RBF-SVM on the raw datasets, indicating useful representations. No noteworthy improvement is made on AWR. The results are almost saturated, since the raw dataset is easier to use for the models. Raw nonlinear classification therefore remains competitive on AWR, where the raw RBF-SVM reaches 0.98 with 100% labels, level with TS2Vec and above T-Loss.

7.2 Whether the effect of SSL becomes clearer with only 10% labels

- FordA: Logistic regression improves massively for FordA – similar absolute improvements compared to 100% case (+0.40 against +0.43 on FordA). RBF-SVM on 10% improves much more than on the full label set (+0.25 against +0.10) and indicates useful representations.
- AWR: No clear effect. With logistic regression T-Loss is about 0.01 below and TS2Vec about 0.01 above the raw baseline in both regimes, within the seed spread. The RBF-SVM cells at 10% use untuned defaults for every representation.
- Epilepsy: Improves logistic regression approx. as much as on the full labels set (+0.30 against +0.33). Improves RBF-SVM more than on the full label set (about +0.16 against +0.12).

The effect of SSL becomes clearer with 10% labels for the RBF-SVM, but not for logistic regression, the principal probe, where the gain is about the same in both regimes.

7.3 Stability of conclusions across the paired seeds

Stability of conclusions for 100% label sets is high. Stability at 10% labels for the FordA dataset is comparable, while the stability is lower for the other datasets. This makes sense, since there will be higher variance in the covered cases in smaller label sets, and each encoder is additionally different.

7.4 Role of dataset size, dimensionality and number of classes

The size and number of classes of FordA most likely causes the high stability for the 10% labels set – it is simply more likely that the classes are well covered among the 10% labels. The opposite is the case for AWR: it is smaller and has many classes, causing it to display lower stability. Epilepsy is between FordA and AWR in terms of labels per class.

While the dimensionalities of the datasets are wildly different, it cannot be assessed what the effect of this is. A regression fitted on a high-dimensional dataset with more features than labels will have multiple perfect fits, letting regularization choose. This does however not explain the results, since AWR has the worst ratio in this aspect, yet the best raw results. The structure and not the dimensionality of the dataset is deemed the deciding factor.

7.5 Whether continuous masking improves accuracy, robustness, both or neither

Neither is improved.

7.6 What the TF-C bearing experiment shows about transfer between operating conditions

- Nothing about transfer. The encoder collapsed in pretraining and carries no information from FD-A, so the run cannot say whether features learned under one operating condition help under another.
- It is a finding about reproducibility: the released execution path does not reproduce the reported result for this case.

7.7 Implications for future multichannel industrial sensor data

- If the datasets are not trivially easy to classify or perform regression on, SSL methods can be used for improving performance of classifiers when labels are sparse, which they often are in industrial cases.
- The size and number of classes heavily influence the stability.
- Transfer learning from TF-C might be useful, but since the results could not be recreated, confidence is low.

8 Experimental rules: compliance notes

- Official test sets preserved as held-out data: Yes. The test split is used only for the final evaluation, never for pretraining, scaling or tuning. The exception is Part F, where the supplied TF-C code evaluates the test set after every epoch.
- Test inputs not used for SSL pretraining in the required experiments: Yes. All encoders are pretrained on the training split only.
- Normalisation, imputation and feature transforms fitted on training data only: Yes. The per-channel scaler and the representation scaler are fitted on the training split. The data loaders of the T-Loss checkout, which normalise on train and test combined, are not used.
- 10% label subsets stratified, indices saved: Yes. The indices are in `results/label_subsets/`, one file per dataset and seed.
- Same seeds, label subsets, corruptions and probe settings in paired comparisons: Yes. Every method uses the same three seeds, label subsets, probe settings and corrupted test sets.
- Tuning on a training-only validation split or cross-validation: Yes. Stratified cross-validation on the training labels; cells that cannot be tuned use fixed defaults.
- Encoder frozen in Parts C–E; Part F follows the supplied TF-C downstream procedure: Yes in Parts C–E: representations are extracted once and the probes are trained on them. In Part F the supplied procedure fine-tunes the encoder.
- Configurations, software versions, runtime and hardware recorded: `src/config.yaml` holds the configuration, `results/runs/` the runtime and device of each pretraining run, and `results/tfc/execution_record.json` the software versions and hardware.
- Implementation departures from the official repositories: T-Loss: an import shim for Python 3.12, `fit_encoder` instead of `fit`, and our own train-only scaling instead of its loaders. TS2Vec: `mask_mode` set after construction, and representation scaling before both probes. TF-C: the `np.float` alias, directory links for the datasets, and `--logs_save_dir`. No checkout file is modified.

9 Limitations and conclusions

9.1 Limitations

- Only three seeds: small differences in results will likely not be significant in this study.
- The datasets differ in size, channels, lengths, classes, and structure. The number of dimensions is too many for a study based upon three datasets to isolate any single factor.
- Only one corruption type and fill convention was used.
- The 10% labels set results from AWR are untuned.
- TF-C was not run with the 2022 library version.
- TF-C only uses the first channel of multivariate data.

9.2 Conclusions

Advantages of SSL Representations of time series achieved using SSL methods can improve results of both linear and non-linear classification methods in both full and sparse label sets. In general, SSL might be useful when the similarities between series belonging to the same class are non-trivial and/or non-linear. Since the representations in this study improved results in both the 100% and 10% label regimes for FordA and Epilepsy (not AWR due to already high performance by the logistic probe), SSL addresses the problem which motivates it – sparsely labelled datasets need methods which can exploit the inherent structure even with missing labels to improve classification results.

Disadvantages of SSL The usefulness depends on the size and number of classes of the datasets, and the structure of the time series. If the datasets contain a structure which is easily exploited by a simpler model, SSL methods might not be worth the complexity and time.

Some SSL methods require substantial time to train, the interpretability of the representations/features is compromised, and care must be taken not to destroy or alter important discriminating information when creating augmentations for pretext tasks. Pretext tasks were, however, in this study not found to produce significantly different results.

Transfer learning Training of TF-C collapsed, and transfer learning, based upon this method, was therefore not demonstrated successfully in this study. How transfer learning compares in results and therefore viability in the face of sparsely labelled datasets, is inconclusive based upon this assignment.

10 Per-seed results

Table 10 shows the test accuracy per seed for logistic regression, the principal probe. All other per-seed results are in the machine-readable files in `results/metrics/`:

- `baselines_raw.csv`: Part B, both probes, accuracy and macro-F1 (36 rows).
- `ssl_probe.csv`: Part C, both methods and both probes (72 rows).
- `corruption.csv`: Part D, clean and corrupted scores for all representations and both probes (54 rows).
- `mask_ablation.csv`: Part E, both maskings (12 rows).

Each row also holds the selected hyperparameters and the number of folds used.

Dataset	Input	Labels	Seed 42	Seed 123	Seed 456
FordA	Raw	100%	0.4924	0.4886	0.4955
FordA	Raw	10%	0.4962	0.5098	0.5114
FordA	T-Loss	100%	0.9227	0.9129	0.9333
FordA	T-Loss	10%	0.9083	0.9053	0.9159
FordA	TS2Vec	100%	0.9258	0.9205	0.9356
FordA	TS2Vec	10%	0.9121	0.9068	0.9174
AWR	Raw	100%	0.9733	0.9733	0.9733
AWR	Raw	10%	0.8533	0.8167	0.8133
AWR	T-Loss	100%	0.9700	0.9700	0.9567
AWR	T-Loss	10%	0.8233	0.8600	0.7600
AWR	TS2Vec	100%	0.9800	0.9833	0.9867
AWR	TS2Vec	10%	0.8567	0.8500	0.8067
Epilepsy	Raw	100%	0.6522	0.6232	0.6232
Epilepsy	Raw	10%	0.3913	0.4058	0.5217
Epilepsy	T-Loss	100%	0.9638	0.9783	0.9638
Epilepsy	T-Loss	10%	0.7609	0.7536	0.7174
Epilepsy	TS2Vec	100%	0.9565	0.9710	0.9710
Epilepsy	TS2Vec	10%	0.7754	0.6812	0.7536

Table 10: Test accuracy per seed with logistic regression, on the raw signal and on the two representations.